Course Discussions

Solution: Merge Two Sorted Lists (easy)

Problem Statement

Given the head of two sorted linked lists, `l1` and `l2`.

Return a new sorted list created by merging together the nodes of the first two lists.

Examples

1. **Example 1**:
    - **Input**:
      ```
      [1, 3, 5]
      [2, 4, 6]
      ```
    - **Expected Output**:
      ```
      [1, 2, 3, 4, 5, 6]
      ```
    - **Justification**: Merging the two sorted linked lists, node by node, results in a single sorted linked list containing all elements from both input lists.
2. **Example 2**:
    - **Input**:
      ```
      [2, 4, 6]
      [1, 3, 5]
      ```
    - **Expected Output**:
      ```
      [1, 2, 3, 4, 5, 6]
      ```
    - **Justification**: Both lists are in ascending order; merging them node by node in ascending order gives us the sorted linked list with all elements.
3. **Example 3**:
    - **Input**:
      ```
      [1, 2, 3]
      [4, 5, 6]
      ```
    - **Expected Output**:
      ```
      [1, 2, 3, 4, 5, 6]
      ```
    - **Justification**: As the first list contains all smaller elements, combining them results in a new list with elements from the first list followed by the second one.

Solution

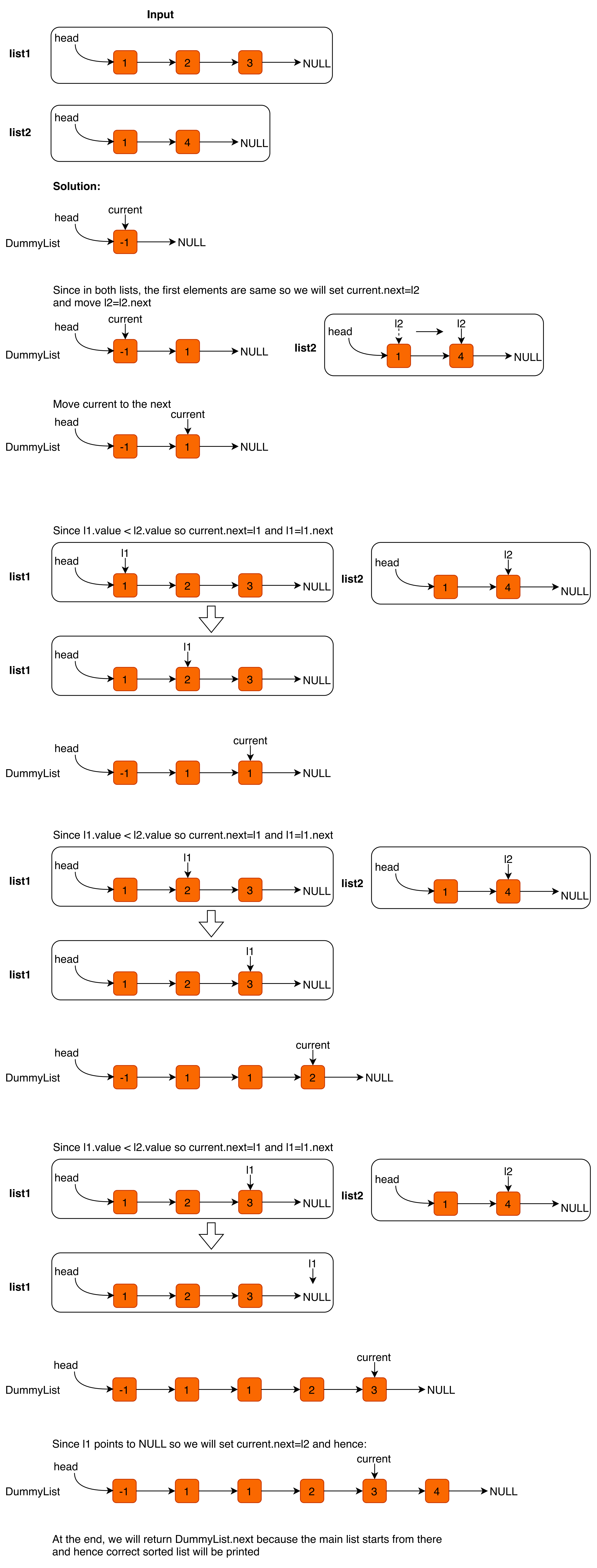
To solve the problem, iteratively compare the nodes of the two given lists and merge them into a new sorted list. Start with a placeholder node as the head of the merged list. Compare the current nodes of both lists; whichever node has the smaller value gets appended to the merged list. Then, move forward in the list that had the smaller element. Repeat this process until one of the lists is fully traversed. At this point, simply append the remaining elements of the other list to the merged list. The placeholder head is then skipped, and the next node is returned as the head of the new, sorted merged list.

1. **Initialization**:
    - Start by initializing two pointers, `l1` and `l2`, at the heads of the first and second lists respectively.
    - Create a dummy node to keep track of the head of the merged list, and a `current` pointer to build the new list.
2. **Node Comparison and Merging**:
    - Compare the values at `l1` and `l2`. Append the node with the smaller value to the `current` pointer.
    - Move the pointer (`l1` or `l2`) whose node has been used forward.
3. **List Traversal**:
    - Continue this process of comparing, appending, and moving pointers until you reach the end of one of the lists.
    - Once one of the lists is exhausted, append the remaining nodes from the other non-empty list to `current`.
4. **Final Output**:
    - After traversing both lists, the `current` pointer will have a sorted merged list attached to it. Return the `next` of the dummy node as the head of the merged list.

Algorithm Walkthrough

- Start with the first nodes of the two lists, compare them and attach the smaller one to `current`.
- Move forward in the list from which the node was taken.
- Continue this process, comparing nodes and appending the smaller one to `current`.
- If one of the lists is exhausted, append the remaining nodes from the other list.
- Return the `next` of the dummy node as the output.

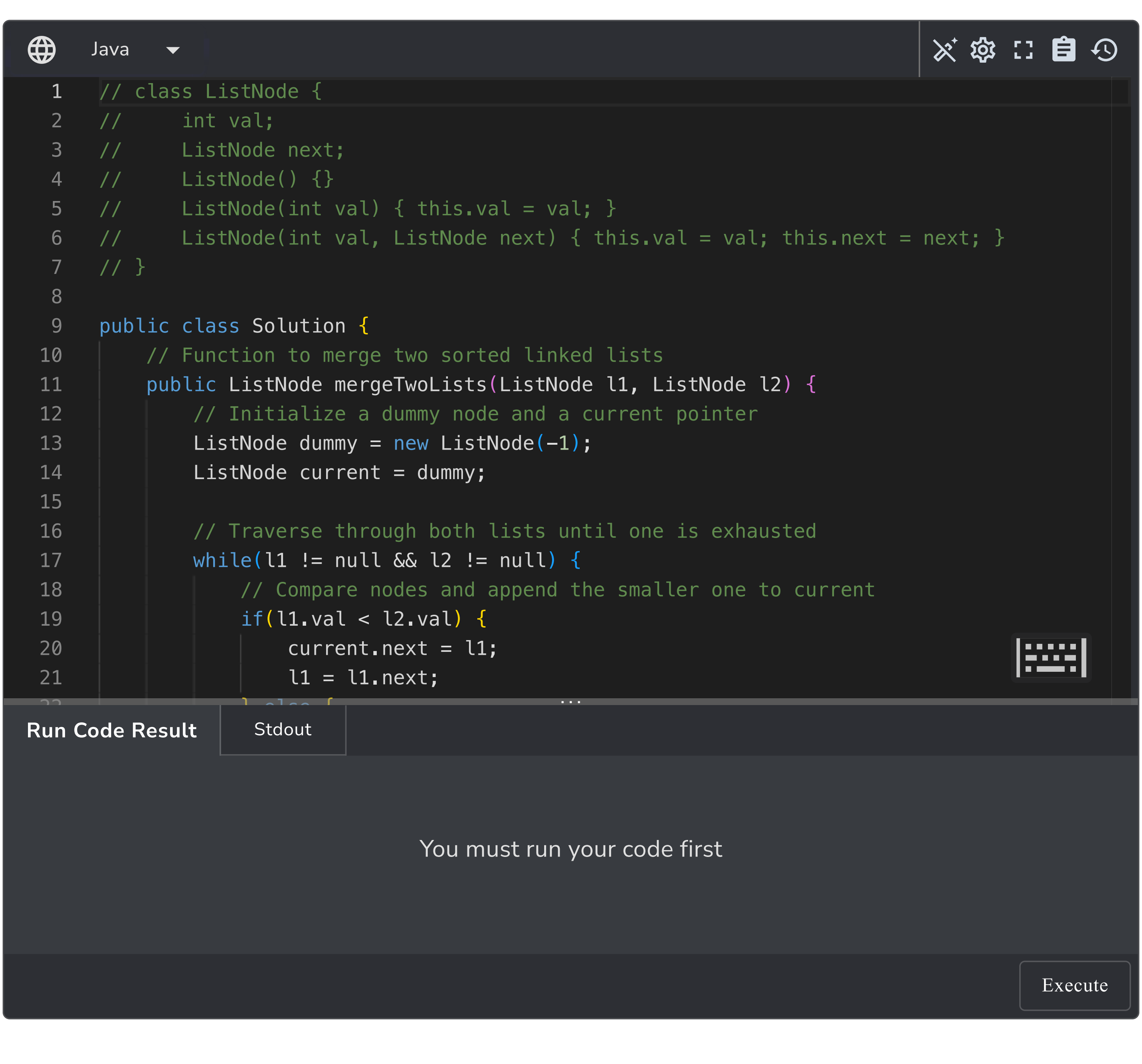
Here is the visual representation of the algorithm:


Input
list1
head
1
2
3
NULL
list2
head
1
4
NULL
Solution:
DummyList
head
current
-1
NULL
Since in both lists, the first elements are same so we will set current.next=l2 and move l2=l2.next
l2
l2
Move current to the next
Since l1.value < l2.value so current.next=l1 and l1=l1.next
l1
Since l1 points to NULL so we will set current.next=l2 and hence:
At the end, we will return DummyList.next because the main list starts from there and hence correct sorted list will be printed

Merge Two Sorted Lists

Code

Here is the code for this algorithm:



```java
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

public class Solution {
    // Function to merge two sorted linked lists
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        // Initialize a dummy node and a current pointer
        ListNode dummy = new ListNode(-1);
        ListNode current = dummy;

        // Traverse through both lists until one is exhausted
        while(l1 != null && l2 != null) {
            // Compare nodes and append the smaller one to current
            if(l1.val < l2.val) {
                current.next = l1;
                l1 = l1.next;
```

Time Complexity:

- **O(n + m)**: The time complexity of merging two sorted linked lists is linear, where `n` and `m` are the lengths of the two input lists. In each step, we're deciding which node (from the two lists) with the smaller value should be added next to the merged list. We continue this process until one of the lists is empty, and then append the remaining list. As a result, every node is visited once, which leads to O(n + m) time complexity.

Space Complexity:

- **O(1)**: The space complexity is constant since we are not using any additional data structures that scale with input size. We're only using a few extra variables to keep track of the current node and the head of the merged list. All other nodes are part of the input or output, and do not count towards the space complexity. The input lists are being used to construct the output list in-place, so the algorithm is very space-efficient.

← Previous
Problem 3: Merge Two Sorted Lists (Easy)

Next →
Problem 4: Find If Doubly Linked List Is A Palindrome (Easy)

☐ Mark as Completed